



Jeżeli szykujesz się na przygodę z sympatycznym gadem, to nie będzie to takie proste. Nazwa Python pochodzi bowiem od serialu "Latający cyrk Monty Pythona", którego wielkim fanem jest twórca tego języka, holender Guido van Rossum. W czasie nauki, bardziej niż na styczność z wężem, nastaw się zatem na dozę brytyjskiego humoru.

Środowisko programistyczne **Python 3 + edytor IDLE.**



Pracę z instrukcją warto rozpocząć po obejrzeniu poniższego filmu:

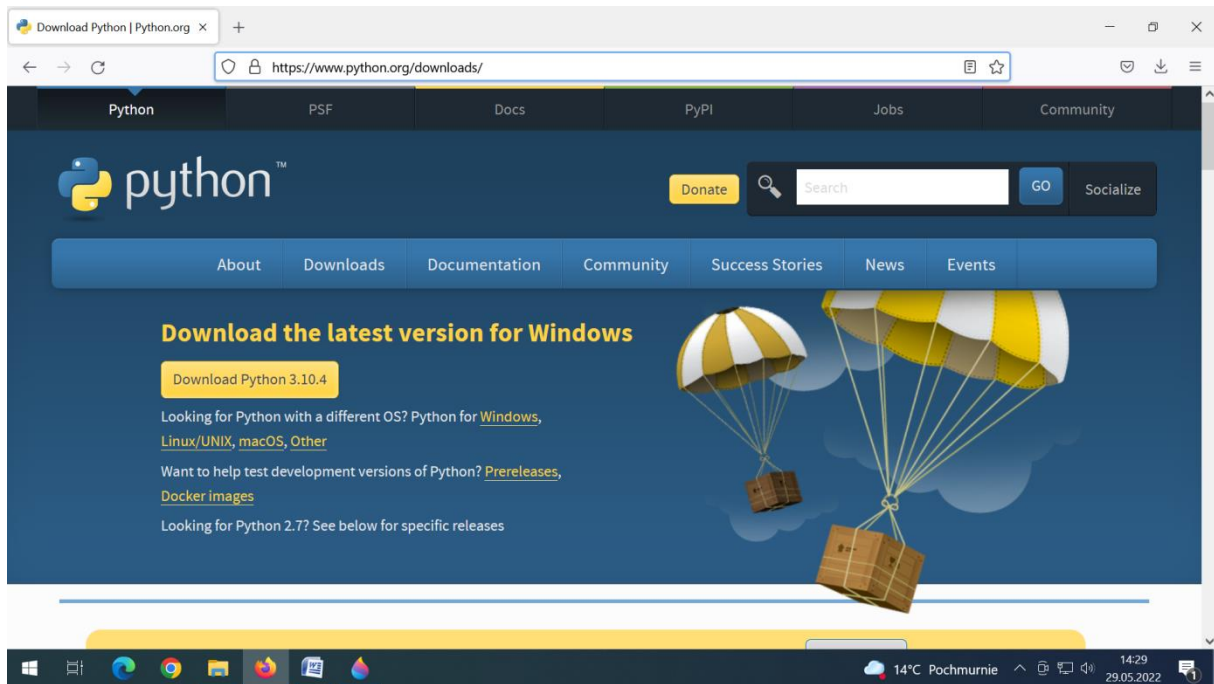
Kurs Python : 2. Instalacja oprogramowania

<https://www.youtube.com/watch?v=STawyS89fgM>

IDLE umożliwia tworzenie programów w języku Python, zapisywanie i uruchamianie. Zintegrowane środowisko programistyczne IDLE zawiera **powłokę Pythona(Shell)**, **edytor kodu źródłowego** i **interpreter** oraz inne narzędzia wspomagające programowanie.

Środowisko programistyczne języka Python możemy zainstalować ze strony:

<https://www.python.org/downloads/>

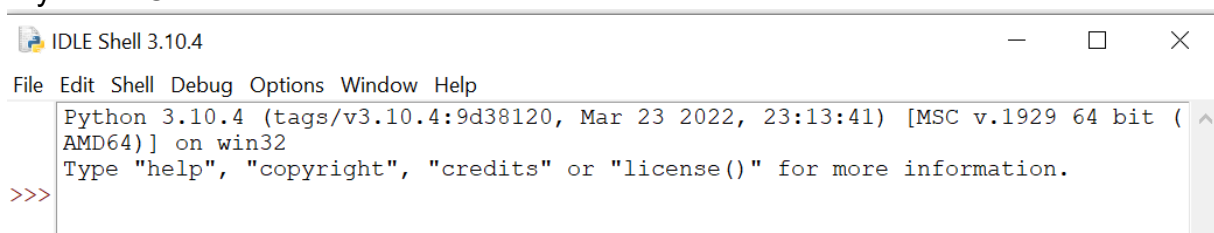


Aby uruchomić IDLE, należy z menu **Start** wybrać program **Python 3.10/IDLE (Python 3.10 64-bit)**. Otworzy się okno programu powłoka Pythona (Python Shell). Okno powłoki Pythona umożliwia pracę w dwóch trybach:

- interaktywnym;
- skryptowym.

Pisanie poleceń w trybie interaktywnym:

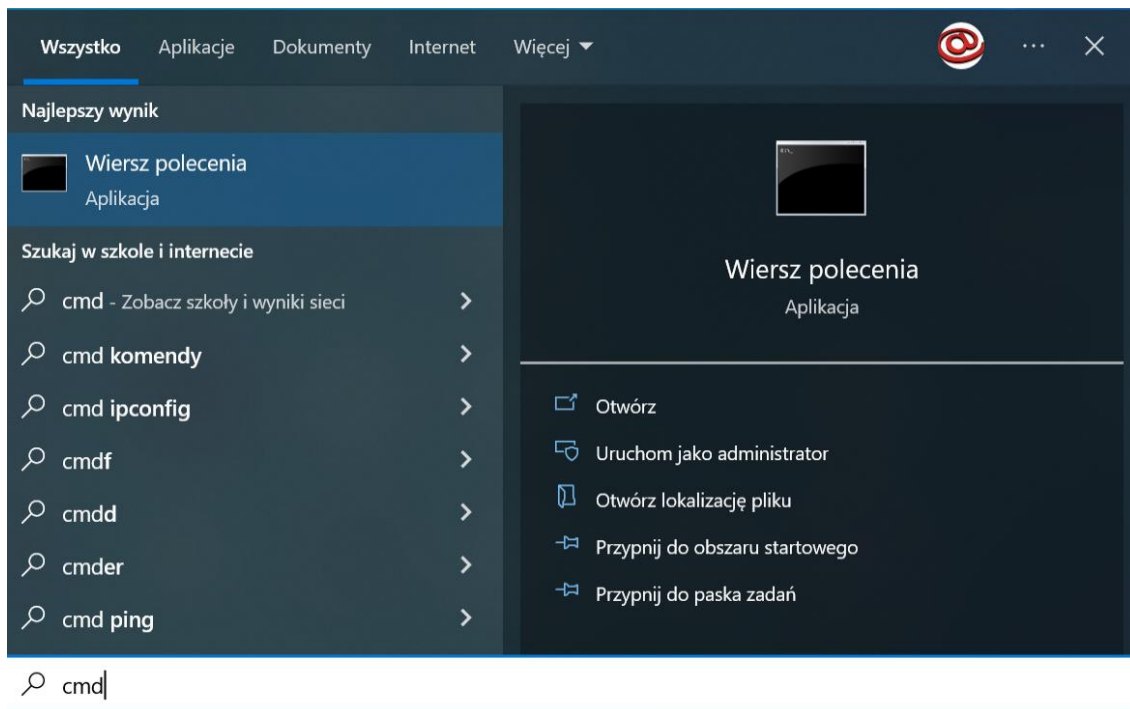
Powłoka Pythona uruchamia się w trybie interaktywnym. Po znaku zachęty **>>>** widzimy migający kursor. W tym miejscu możemy napisać instrukcję lub wyrażenie. po naciśnięciu klawisza **Enter** instrukcja zostanie wykonana, a wyrażenie obliczone. Trybem interaktywnego używaliśmy się także w innych językach np: Logo i Scratch czy też w C++.



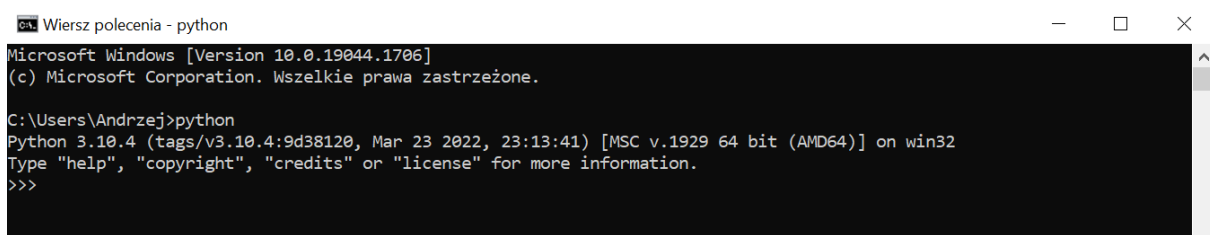
Konsola pozwala nam wykonać kod i sprawdzić jego działanie.

Ćwiczenie 1:

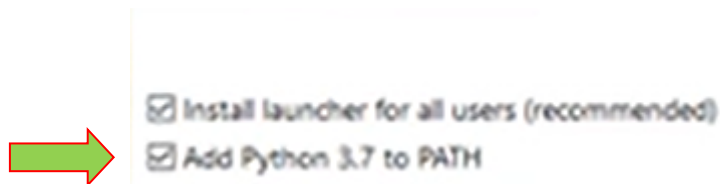
Teraz sprawdźmy czy możemy uruchomić Python-a z poziomu Windowsowego wiersza poleceń. Aby to zrobić klikamy po prostu na przycisk **START**/wpisujemy komendę **cmd** i wybieramy **wiersz polecenia**.



W wierszu polecenia wpisujemy komendę `python` i wciskamy ENTER.



Jeśli wyświetli Ci się komunikat o wersji Pythona to wszystko w porządku. W przypadku komunikatu o błędzie, prawdopodobnie zapomniałeś zaznaczyć ptaszka w oknie dodatnia ścieżki umożliwiającej uruchomienie Python-a z Windowsowego okna poleceń. Ponowna instalacja powinna pomóc.



Teraz sprawdźmy działanie trybu interaktywnego:

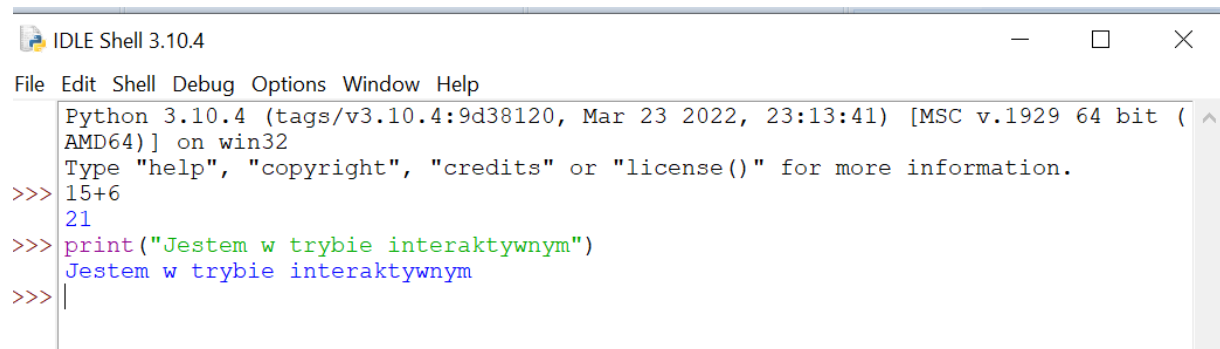
Ćwiczenie 2:

Uruchom środowisko IDLE.

W oknie powłoki Pythona sprawdź działanie trybu interaktywnego. Wpisz po znaku zachęty:

15 + 6 i naciśnij Enter,

print("Jestem w trybie interaktywnym") i naciśnij Enter.



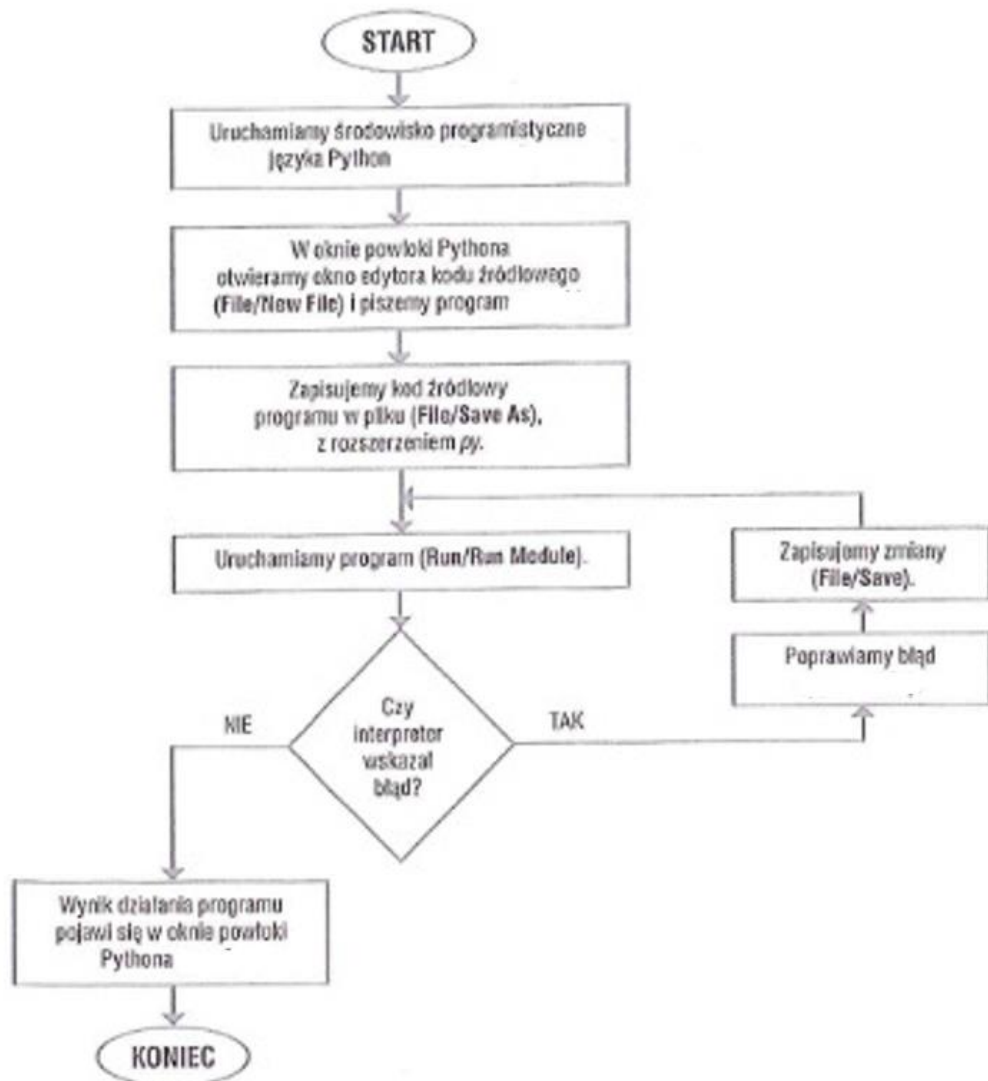
```
IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> 15+6
21
>>> print("Jestem w trybie interaktywnym")
Jestem w trybie interaktywnym
>>> |
```

Pisanie programów w trybie skryptowym

W językach interpretowanych, takich jak Python, możemy napisać program składający się z wielu poleceń (tworzący tzw. skrypt), zapisać go w pliku tekstowym, a następnie uruchomić, używając interpretera. Mówimy wówczas, że pracujemy w trybie skryptowym.

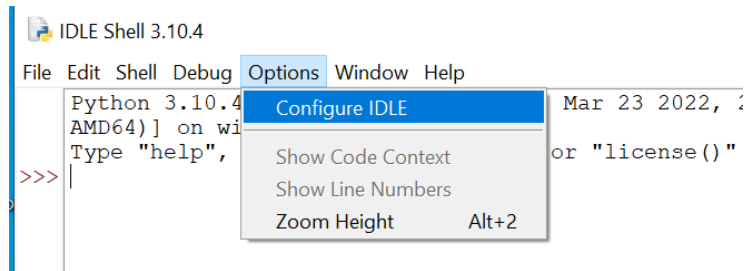
Do trybu skryptowego możemy przejść z trybu interaktywnego z okna powłoki Pythona. Program piszemy w edytorze kodu źródłowego, zapisujemy w pliku na dysku, a następnie uruchamiamy (poprzez wywołanie interpretera Pythona).

W języku Python możemy wyróżnić etapy tworzenia programu pokazane poniżej:



Podstawowe zasady pisania programu w języku Python

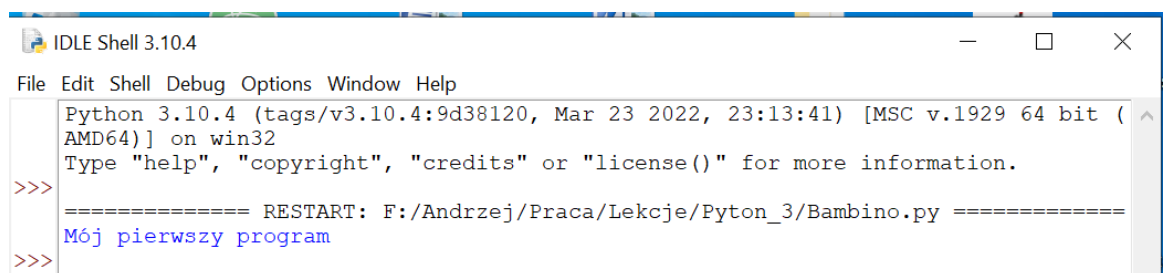
- W programowaniu w języku Python, jak w każdym języku, bardzo istotna jest prawidłowa kolejność zapisu poleceń.
- Instrukcje języka Python, podobnie jak innych języków programowania, realizują takie czynności, jak: wprowadzanie danych, wyprowadzanie wyników, wykonywanie obliczeń, sprawdzanie warunków czy powtarzanie operacji.
- Bloki instrukcji wyodrębnia się inaczej niż w innych językach. Nie stosuje się nawiasów ani słów kluczowych, tylko wcięcia, składające się z przynajmniej jednej spacji (**przyjęte jest wcięcie składające się z czterech spacji**).
- Komentarze rozpoczyna się znakiem #. Tekst po znaku # do końca wiersza jest ignorowany przez interpreter.
- Na zrzucie poniżej pokazałem jak można zmieniać rozmiar czcionki:



Ćwiczenie 3

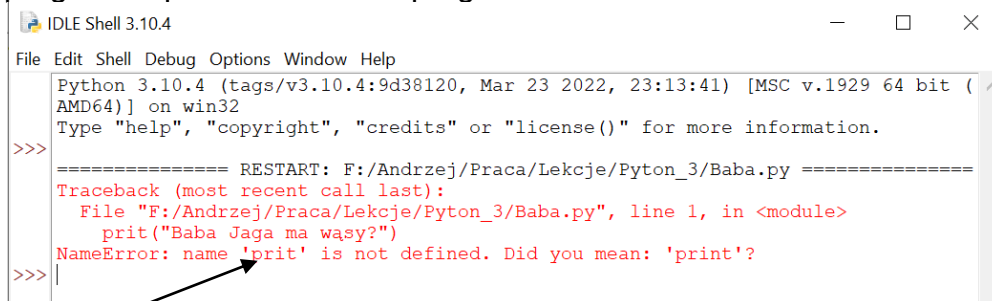
Tworzenie programu w języku Python

1. Utwórz nowy plik źródłowy i napisz program wyświetlający na ekranie napis „Mój pierwszy program”.
2. Zapisz program w pliku pod nazwą Bambino.
3. Uruchom program.
4. Jeśli interpreter wykrył błędy, popraw je. Zapisz plik pod tą samą nazwą i ponownie uruchom.



W trakcie wykonywania programu interpreter rozpoznaje polecenia języka Python i zamienia je na instrukcje wewnętrznego języka procesora. Po wykonaniu programu wyniki pojawiają się na wybranym urządzeniu zewnętrznym, np. na ekranie monitora (tu w oknie powłoki Pythona).

W odróżnieniu od kompilatora, interpreter próbuje wykonać program instrukcja po instrukcji. Jeżeli w programie wystąpi błąd, interpreter przerwie wykonanie programu i wyświetli komunikat o błędzie. Należy poprawić wskazany błąd, ponownie zapisać program w pliku i uruchomić program.



Komunikat interpretera informuje o nieznanej nazwie prit. Podpowiada prawidłową składnię polecenia: print.

Stosowanie zmiennych

Chcemy obliczyć iloczyn dwóch liczb całkowitych. Jak napisać w języku Python program umożliwiający wprowadzenie dwóch liczb całkowitych z

klawiatury, obliczający ich iloczyn i wyprowadzający wynik obliczeń na ekran?

W wielu językach programowania (np. w języku C++) przed użyciem zmiennej należy ją zadeklarować, czyli określić jej nazwę i typ. **W języku Python nie deklaruje się zmiennych.** Nie musimy wcześniej określać typu zmiennej, czyli rodzaju przechowywanych w niej danych (np. liczby całkowite, rzeczywiste, znaki).

Zasady dotyczące nazw zmiennych w języku Python

1. Wielkie i małe litery w nazwach są traktowane odmiennie (np. suma i Suma będą oznaczać różne zmienne). Pisząc program, należy zwracać uwagę na poprawne używanie małych i wielkich liter.
2. W nazwach zmiennych powinno się używać liter, znaku podkreślenia i cyfr. Nazwa nie może zaczynać się od cyfry. Przyjęte jest stosowanie małych liter i niestosowanie polskich liter.
3. W nazwach zmiennych nie wolno stosować spacji. W przypadku nazw kilkuczłonowych zamiast spacji stosujemy znak podkreślenia.
4. Wskazane jest tworzyć nazwy zmiennych, które określają znaczenie danej zmiennej, np. suma, liczba elementów.

Interpreter określa typ zmiennej na podstawie rodzaju danych, jaki jej przypiszemy. **Nie ustanawia jednak typu zmiennej na stałe**, ponieważ przy kolejnym przypisaniu może się on zmienić. Na przykład, jeśli na początku programu przypiszemy zmiennej liczba wartość całkowitą (27), to jej typ jest całkowity. Jeśli jednak w dalszej części programu przypiszemy jej wartość rzeczywistą (np. 81,8, którą w języku Python zapisujemy z kropką: 81.8), to zmienna liczba będzie typu rzeczywistego.

Nadawanie zmiennym wartości

Zmiennej stosowanej w programie możemy nadać konkretną wartość za pomocą **instrukcji przypisania**. instrukcji przypisania zmiennej podanej po lewej stronie instrukcji zostanie obliczona przez komputer wartość wyrażenia znajdującego się po prawej stronie instrukcji.

Instrukcja przypisania	zmienna = wyrażenie
------------------------	---------------------

Wyrażenie to m.in.: konkretna wartość, zmienna, wyrażenie algebraiczne.

Przykład	Opis
rok = 2022	Zmiennej rok przypisujemy wartość, która jest liczbą całkowitą, czyli zmienna rok będzie typu całkowitego (int).
x = 6.24	Zmiennej x przypisujemy wartość, która jest liczbą rzeczywistą, czyli zmienna x będzie typu rzeczywistego (float).

<code>z = y</code>	Zmiennej z przypisujemy wartość zmiennej y.
<code>obwod = 2 * a + 2 * b</code>	Zmiennej obwod przypisujemy wartość wyrażenia $2 \cdot a + 2 \cdot b$; jeśli wynik obliczeń będzie liczbą całkowitą, interpreter określi dla zmiennej obwod typ całkowity; jeśli wynik będzie liczbą dziesiętną, interpreter określi typ zmiennej jako rzeczywisty.
<code>liczba = 12</code> <code>liczba = 35.4</code>	Zmiennej liczba przypisujemy wartość 12 (która jest liczbą całkowitą), a w drugiej instrukcji - wartość 35,4 (która jest liczbą rzeczywistą); poprzednia wartość zmiennej liczba zostanie usunięta, a interpreter określi dla zmiennej liczba typ rzeczywisty.

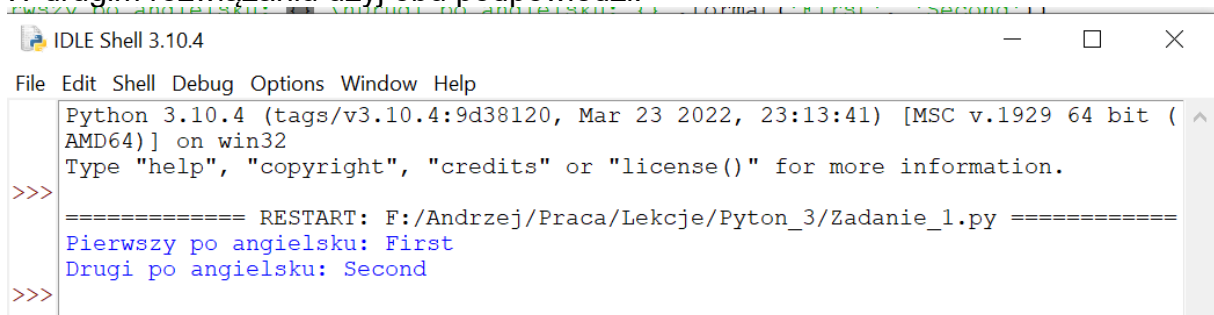
Zadanie 1

Umiesz już wykonywać napisy w Pythonie. Tym razem zadanie wymagające poszukania odpowiedzi. Jak uzyskać poniższe napisy za pomocą jednego polecenia „print”?

Zadanie rozwiąż dwoma sposobami:

W pierwszym rozwiązaniu użyj tylko pierwszej podpowiedzi.

W drugim rozwiązaniu użyj obu podpowiedzi.



```

Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: F:/Andrzej/Praca/Lekcje/Pyton_3/Zadanie_1.py =====
Pierwszy po angielsku: First
Drugi po angielsku: Second
>>>

```

Podpowiedzi:

- 1.Podobnie jak w C++ możemy użyć: `\n` – znaku nowej linii;
- 2.W nowym formatowaniu miejsca wstawiania oznaczamy przez klamry {}, a napis i argumenty łączymy przez `.format()`.

Zadanie 2

Teraz możemy wykonać zadanie 1 z pierwszej części instrukcji do C++.

Napisz program opisujący zakresy wartości przedstawionych w tabeli (Zakres wartości typów danych – cz.2 instr. C++) umieszczonej w instrukcji w formie tabelarycznej (bez ramek). Wpisujesz z tabeli numery wierszy nr_z_dziennika+4. Np. Uczeń o numerze 7 w dzienniku wpisuje 7 ,11, 15, 19, 23, 27, 3.

```

IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: F:\Andrzej\Praca\Lekcje\Python_3\Zadanie_2.py =====
Lp.      Typ      Typowy rozmiar w bajtach      rozmiar
7        signed short      2 bajty      od -32768 do 32767
11       Float            4 bajty      3.4 E- 38 do 3.4e+38
15       unsigned short int  2 bajty      0 do 65535
19       unsigned long long  8 bajtów     Od 0 do 18446744073709551615
23       Long              4 bajty      -2147483648 do 2147483647
27       Signed long long   8 bajtów     Od -9223372036854775808 do 92233720368547758
3        unsigned short     2 bajty      0 do 65535
>>>

```

Zmiennej stosowanej w programie możemy również nadać wartość **za pomocą instrukcji przypisania**, wprowadzając wartość z klawiatury w trakcie działania programu. W tym celu stosujemy instrukcję wejścia — funkcję **input()**.

Wprowadzenie danych z klawiatury ---- zmienna = input (tekst_zachęty)

Argumentem funkcji input() (podanym w nawiasach) jest tekst (ciąg znaków, łańcuch). Tekst ten pojawi się na ekranie jako swego rodzaju „zaproszenie” do wpisania ciągu znaków z klawiatury. Jeśli użyjemy funkcji input() w instrukcji przypisania, wpisany przez użytkownika ciąg znaków zostanie zapamiętany w zmiennej podanej po lewej stronie.

Aby dana wprowadzona za pomocą instrukcji input() została zapamiętana jako : liczba, musimy dodać instrukcję, która zamieni ciąg znaków na liczbę: int() — w przypadku liczb całkowitych, float() — w przypadku liczb rzeczywistych.

Przykład	Opis
X= input("Wprowadź liczbę: ")	Jeśli wpiszemy z klawiatury liczbę 123, w zmiennej X zostanie zapamiętany ciąg znaków „123” a nie liczba 123.
Y= int(input("Wprowadź liczbę: "))	Jeśli wpiszemy z klawiatury liczbę 123, w zmiennej Y zostanie zapamiętana liczba całkowita 123.
srednia = float(input("Podaj średnią ocen : "))	Jeśli wpiszemy z klawiatury liczbę 3.3, w zmiennej srednia zostanie zapamiętana liczba rzeczywista 3,3.

miasto = input("Wprowadź nazwę miasta: ")

Jeśli wpiszemy z klawiatury nazwę miasta „Gdansk”, w zmiennej zostanie zapamiętany ciąg „Gdansk”.

Wykonywanie obliczeń

W języku Python obliczenia możemy zapisywać, korzystając z instrukcji przypisania. W obliczeniach stosujemy podstawowe operatory arytmetyczne:

+ (dodawania),
 - (odejmowania),
 * (mnożenia),
 // (dzielenia całkowitego),
 / (dzielenia zmiennoprzecinkowego)
 % (reszty z dzielenia dwóch liczb całkowitych).
 **potęgowanie:

Przykład wyrażenia z operatorem potęgowania ****** :

```
print( 2 ** 3 )
```

```
8
```

W ten sposób obliczyliśmy wartość liczby 2 podniesionej do trzeciej potęgi.

Wyprowadzanie komunikatów i wyników na ekran monitora

Dotychczas wyprowadzaliśmy komunikaty na ekran monitora, używając polecenia print. Za pomocą tego polecenia można również wyprowadzać wyniki obliczeń.

Wyprowadzanie wyników i komunikatów na ekran --- print (wartość)

W języku Python print() jest funkcją, która wyświetla na ekranie wartość zapisaną w nawiasach i umieszcza kursor w nowym wierszu.

Wartość umieszczana wewnątrz nawiasów to np. zmienna, wyrażenie, a także napis. Napis umieszczany w programie, który ma zostać wyświetlony na ekranie monitora, musi być ujęty w górne cudzysłowy podwójne **" "** lub pojedyncze **' '**. Funkcja print() pozwala łączyć wyświetlanie napisów, wyrażeń i zmiennych. Poszczególne wartości, występujące jako parametry funkcji print(), należy rozdzielić przecinkami.

Przykład łączenia napisów i zmiennych (zmienna a została uprzednio wprowadzona):

```
print ("Dla boku o długości ", a, "pole kwadratu wynosi: ", a *a)
```

Zadanie 3

Wykonaj program działania kalkulatora, który po wczytaniu dwóch liczb naturalnych dodatnich w jednej instrukcji poda wynik:

- Suma liczb
- Różnica liczb
- Iloczyn liczb
- Iloraz liczb
- Reszta z dzielenia

Zmienne a i b zapisz z trzema literami nazwiska np.: a_raw.

Wyświetlanie wyniku ilorazu ogranicz do dwóch cyfr po przecinku.

Wygląd ekranu:

Np.

Podaj a=

Podaj b=

Wynik działania programu(wygląd ekranu):

suma=5

różnica=-1

iloczyn=6

iloraz=0,67

reszta z dzielenia=2

```
==== RESTART: F:\Andrzej\
Podaj liczbe a: 2
Podaj liczbe b: 3

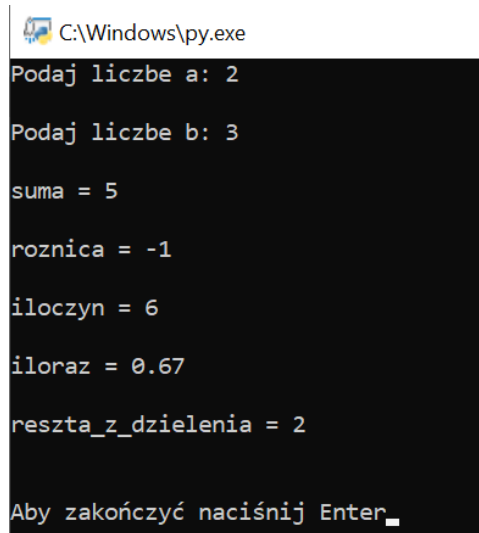
suma = 5

roznica = -1

iloczyn = 6

iloraz = 0.67

reszta_z_dzielenia = 2
>>> |
```



```
C:\Windows\py.exe
Podaj liczbe a: 2
Podaj liczbe b: 3

suma = 5

roznica = -1

iloczyn = 6

iloraz = 0.67

reszta_z_dzielenia = 2

Aby zakończyć naciśnij Enter_
```

W większości języków programowania, w tym w języku Python, część dziesiętną liczby zapisujemy po kropce, a nie po przecinku, jak na matematyce. W ten sam sposób liczby dziesiętne są również wyświetlane na ekranie.

Program możemy również uruchomić, klikając dwukrotnie nazwę pliku z zapisanym programem w Eksploratorze plików — otworzy się okno, w którym zobaczymy wynik działania programu. Aby program został wykonany, niezbędny jest interpreter Pythona.

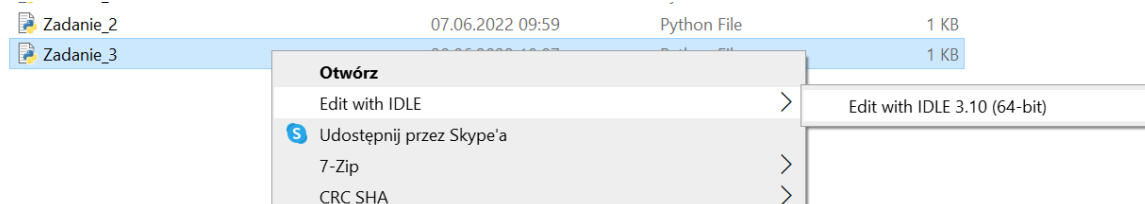
Aby okno programu nie zamknęło się automatycznie po wyświetleniu napisu, możemy

napisać na końcu programu instrukcję oczekiwania na naciśnięcie klawisza Enter:

```
input("\n\n Aby zakończyć, naciśnij Enter")
```

W tym przypadku wartość zwracana przez funkcję `input()` jest pomijana (nie jest przypisywana żadnej zmiennej). Umieszczenie zapisu oznacza `\n\n` oznacza wstawienie dwóch pustych wierszy przed komunikatem.

Jeśli zamknęliśmy okno powłoki Pythona, a chcemy edytować program, można z menu kontekstowego nazwy pliku w oknie Eksploratora plików wybrać polecenie Edit with IDLE/ Edit with 3.10



Środowisko programistyczne Pythona IDLE udostępnia dwa tryby pracy:

- ❖ **interaktywny**, w którym możemy napisać jedno polecenie i zatwierdzić je klawiszem Enter; jeśli polecenie będzie poprawne, zostanie od razu wykonane,
- ❖ **skryptowy**, w którym możemy napisać program, zapisać go w pliku, a następnie uruchomić; jeśli program będzie poprawny, zostanie wykonany.

Aby utworzyć program w języku Python, piszemy kod źródłowy w edytorze kodu źródłowego, zapisujemy program w pliku i uruchamiamy za pomocą interpretera Pythona.

W języku Python typ wartości, która jest przypisana zmiennej, określa typ tej zmiennej.

Zmiennej możemy nadać wartość za pomocą instrukcji przypisania. **Po lewej stronie znaku = podajemy nazwę zmiennej, a po prawej: - konkretną wartość**

lub

funkcję `input()`, dzięki której użytkownik wprowadzi tę wartość z klawiatury po uruchomieniu programu

lub - **wyrażenie, którego wartość zostanie obliczona w czasie działania programu.**

Zmienne w wyrażeniach

Do zmiennych możesz przypisywać wartości wyrażeń, także takich, w których są inne zmienne, np.:

```
droga = 45
czas = 15
predkosc = droga / czas
print ( predkosc )
3
```

```

droga = 45
czas = 15
predkosc = droga / czas
print ( predkosc )

```

```
3
```

Przykład A

w programie utworzono dwie zmienne: cena oraz vat i przypisano im pewne wartości. Utwórz zmienną o nazwie cena_brutto i przypisz jej wartość będącą iloczynem zmiennych cena oraz vat. Następnie wyświetl wartość tej zmiennej.

```

1 cena = 20
2 vat = 1.23

```

Rozwiązanie:

```

File Edit Format Run Options W
cena = 20
vat = 1.23
cena_brutto = cena*vat
print(cena_brutto)

```

Wynik (wyświetlona wartość zmiennej cena_brutto): 24.6

Wartości różnych typów

Do zmiennych możesz przypisywać wartości różnych typów, m.in.: liczby całkowite, liczby zmiennoprzecinkowe (ułamki), łańcuchy znaków, wartości logiczne (prawda/fałsz),

```

imie = "Romek"
wiek = 12
print( imie, "ma", wiek,
"lat." )

```

```
Romek ma 12 lat.
```

Słownik programisty:

Logiczny typ danych (ang. *boolean*) - typ danych, który składa się z dwóch wartości: True (prawda) i False (fałsz).

Przykład B

w funkcji print() użyto zmiennej imie_kota. Utwórz wcześniej (np. w linii nr 1) taką zmienną i przypisz jej imię **Garfield**.

```

1
2
3 potrawa = "Lasagne"
4 print( imie_kota, "kocha", potrawa )

```

Rozwiązanie

```

1 imie_kota="Garfield"
2 potrawa = "Lasagne"
3 print( imie_kota, "kocha", potrawa )

```

Garfield kocha Lasagne

Zmienne – ćwiczenie

Zadanie 4

w programie użyto funkcji print() oraz kilku zmiennych jako jej argumentów. Nie zmieniając tej instrukcji, utwórz wcześniej odpowiednie zmienne i przypisz im wartości w taki sposób, aby efektem działania programu był komunikat:

Barbam video, sed philosophum non video.

czyli z łaciny na polski: *Brodę widzę, ale filozofa nie widzę*

```

1
2
3
4 print( zdanie1, "video,", zdanie2, "video." )

```

a_Python/Zadanie_4.py
Barbam video, sed philosophum non video.

Funkcje

Funkcja to fragment kodu, który ma wykonać określone zadanie i którego można wielokrotnie używać w programie. W programach możesz korzystać ze standardowych funkcji języka programowania (np. funkcja print()), z własnych funkcji, a także z funkcji napisanych przez innych programistów.

Tworzenie funkcji w Pythonie zaczyna się od słowa kluczowego def. Po nim określa się nazwę funkcji, następnie umieszcza parę nawiasów () oraz dwukropek :

Zestaw instrukcji, który ma wykonywać funkcja umieszcza się w kolejnych liniach, z przesunięciem w prawo, czyli z **wcięciem**. Prosta funkcja może wyglądać np. tak

```

def wyswietl():
    print( "Hi there!" )

```

W ten sposób utworzyliśmy funkcję o nazwie `wyswietl()`, której zadaniem jest wyświetlenie komunikatu **Hi there!**

Wywoływanie funkcji

Efekt działania funkcji jest widoczny dopiero po jej użyciu, czyli **wywołaniu**. Wywołanie funkcji polega na podaniu jej nazwy i ewentualnie argumentów, np.:

```
def wyswietl():
    print( "Hi there!")

wyswietl()

Hi there!
```

Zwróć uwagę, że funkcja została wywołana już w głównej części programu, tzn. **bez wcięcia**. Gdyby wywołanie funkcji (ten fragment o żółtym kolorze) był również wcięty, byłby częścią ciała funkcji - tzn. że funkcja wywoływałaby samą siebie! Taka technika zwana jest **rekurencją**. Na razie nie będziemy jej jednak stosować, zatem jeżeli polecenie polega na wywołaniu funkcji, umieszczaj je już po instrukcjach należących do funkcji, bez wcięcia.

Przykład C

Utwórz funkcję o nazwie `moja_funkcja()`, której zadaniem jest wyświetlenie komunikatu:

Ignoramus et ignorabimus

czyli z łaciny: *Nie wiemy i nie będziemy wiedzieć*

Następnie wywołaj tę funkcję.

```
def moja_funkcja():
    print("Ignoramus et ignorabimus")

a_u_c.py
Ignoramus et ignorabimus
```

Parametry funkcji

Funkcje potrafią dużo więcej niż tylko wykonywać instrukcje. Można do nich także wysłać dane. Aby to zrobić, trzeba w funkcji zadeklarować tzw. parametry. Parametry deklaruje się (czyli tworzy) w nawiasach po nazwie funkcji, np.:

```
def wyswietl( jakis_tekst ):
    print( jakis_tekst )
```

Ten zaznaczony na żółto fragment, to parametr funkcji `wyswietl()`. Parametr jest zmienną lokalną dla funkcji. Tzn. możesz mu nadać dowolną nazwę, tak jak w przypadku zmiennych.

Do parametru możesz przekazać wartość (czyli przypisać mu wartość), wywołując funkcję, np.:

```
def wyswietl( jakis_tekst ):
    print( jakis_tekst )

wyswietl( "Ole!" )
```

Ole!

W wywołaniu funkcji, użyliśmy argumentu Ole!. To znaczy, że taka wartości została przypisana do parametru `jakis_tekst`. Następnie instrukcja:

```
print( jakis_tekst )
```

wyświetliła wartość tego parametru.

Wywołując funkcje, możesz przekazać do ich parametrów dane dowolnego typu, np. łańcuchy znaków, liczby całkowite, liczby zmiennoprzecinkowe, wartości logiczne itd., np.:

```
def pomnoz( liczba ):
    print( liczba * 2 )

pomnoz( 3.5 )
```

7.0

Więcej parametrów

Jeżeli chcesz przekazać do funkcji więcej wartości, musisz zadeklarować (utworzyć) więcej parametrów. Parametry oddzielamy od siebie przecinkami. W wywołaniu funkcji, należy podać tyle argumentów (wartości) ile parametrów ma funkcja. Argumenty - podobnie jak parametry - oddzielamy przecinkami. Np.:

```
print( liczba1 * liczba2 )

pomnoz( 2.5, 4 )
```

10.0

Zadanie 5

Podobnie jak w zadaniu 3 wykonaj program działania kalkulatora, który po wczytaniu dwóch liczb naturalnych dodatnich w jednej instrukcji poda wynik. Tym razem stwórz oddzielne funkcje dla każdego działania.

- Suma liczb
- Różnica liczb
- Iloczyn liczb
- Iloraz liczb
- Reszta z dzielenia

```
= RESTART: D:\Python\Python_gotowy_poczatek\Z
```

```
Podaj liczbe a: 2
```

```
Podaj liczbe b: 3
```

```
suma = 5
```

```
roznica = -1
```

```
iloczyn = 6
```

```
iloraz = 0.67
```

```
reszta z dzielenia = 2
```

```
pierwiastek kwadratowy z liczby a = 1.41
```

```
pierwiastek kwadratowy z liczby b = 1.73
```

```
>>>
```

kwadratowy liczby x można obliczyć jako:

```
x = 16
```

```
pierwiastek = x ** 0.5
```

```
print(pierwiastek) # Wynik: 4.0
```

czyli $\sqrt{16} = 4$

Jeśli pracujesz z większymi zbiorami danych, warto skorzystać z biblioteki **numpy**, która oferuje funkcję `np.sqrt`.

```
import numpy as np
```

```
x = 16
```

```
pierwiastek = np.sqrt(x)
```

```
print(pierwiastek) # Wynik: 4.0
```

Zapis **import numpy as np** w Pythonie oznacza, że importujemy bibliotekę **NumPy** i nadajemy jej alias (skrót) **np**. Dzięki temu zamiast pisać za każdym razem pełną nazwę `numpy`, możemy używać krótszego `np`. Jest to praktyczne i bardzo popularne w społeczności Pythona.

Jeśli podczas kompilacji programu pojawi się komunikat braku zainstalowania biblioteki **NumPy** w Twoim systemie lub w środowisku, w którym aktualnie pracujesz. Python po prostu mówi: *"Chcesz użyć NumPy, ale ja go nigdzie nie widzę"*.

Oto jak to naprawić sytuacji w środowisku IDLE Pythona:

W Pythonie można ograniczyć ilość cyfr po przecinku na różne sposoby, w zależności od potrzeb i kontekstu. Jedną z możliwości jest użycie funkcji **round**:



Zwróć uwagę że **nie musisz** tak jak było to w C++ **deklarować typu zmiennych**. Szczególnie widoczne jest to w przypadku wyświetlania wartości ilorazu liczb.

Użycie operatora potęgowania ******

Pierwiastek to potęga o wykładniku ułamkowym. Pierwiastek

Musisz pobrać tę bibliotekę za pomocą menedżera pakietów pip.

1. Otwórz **Wiersz poleceń (cmd)** lub **PowerShell** (w Windowsie wpisz cmd w menu Start).
2. Wpisz poniższą komendę i naciśnij Enter:

```
pip install numpy
```

teraz program powinien zadziałać poprawnie

```
import numpy as np
x = 16
def pierwiastek():
    return np.sqrt(x)
print(pierwiastek())
```

```
>>> |=====
>>> | 4.0
```

Dlaczego stosuje się aliasy?

- **Czytelność kodu:** Użycie aliasu skraca zapis, co jest szczególnie przydatne przy częstym korzystaniu z funkcji biblioteki.
- **Standaryzacja:** W dokumentacji i przykładach większość użytkowników Pythona używa aliasu np dla NumPy, co ułatwia zrozumienie i współpracę z innymi programistami.

Więcej wywołań

Funkcje tworzy się po to, aby móc z nich wielokrotnie korzystać. Każdą funkcję możesz w programie wywoływać wiele razy, np.:

```
komunikat( 2 + 2 + 2 )
komunikat("Jakieś bzdury")

Czas oczekiwania: 5 min.
Czas oczekiwania: 12 min.
Czas oczekiwania: 6 min.
Czas oczekiwania: Jakieś
bzdury min.
```

Jak widzisz po ostatnim komunikacie, jako argumentu możesz użyć danych dowolnego typu.

Przykład D

Dana jest funkcja dodawanie(). Wywołaj ją dwa razy najpierw z argumentami 10 i 3 a potem z argumentami Świąty i Mikołaj.

```
1 def dodawanie( wartosc1, wartosc2 ):
2     print( wartosc1, "+", wartosc2, "=", wartosc1 + wartosc2 )
```

Uzupełnij przykład aby uzyskać efekt widoczny na zrzucie z ekranu.

```
= RESTART: D:\Python\Python_gotowy_pocza
10 + 3 = 13
Święty + Mikołaj = Święty Mikołaj
```

Funkcja zwraca... wartość

Funkcje potrafią nie tylko odebrać dane, ale także wysłać dane z powrotem. Aby funkcja zwracała wartość, w ciele funkcji musi wystąpić instrukcja **return**. Funkcja zwraca wartość wyrażenia, które jest po instrukcji return. Spójrz na przykład: fragment kodu zaznaczony na zielono przyjmuje wartość fragmentu kodu zaznaczonego na czerwono:

```
def dodaj( x, y ):
    return x + y

print( dodaj( 8, 3 ) )

11
```

Instrukcja wywołująca funkcję (ta zielona), jest zatem wyrażeniem, tzn. ma jakąś wartość (w przykładzie ma wartość 11).

Przyjrzyj się także takiemu przykładowi:

```
def dodaj( x, y ):
    return 0

print( dodaj( 8, 3 ) )

0
```

Sama funkcja z powyższego przykładu jest raczej bezsensowna, ale pokazuje logikę mechanizmu zwracania wartości. Ta funkcja zawsze zwraca wartość 0 (ponieważ po instrukcji return jest wartość 0), zatem niezależnie od przekazanych argumentów, wyrażenie wywołujące funkcję: dodaj(8, 3) przyjmuje wartość 0.

Zwróć uwagę, że skoro funkcja zwraca wartość, to możesz z tą wartością coś zrobić, np. wyświetlić ją lub użyć w obliczeniach. Np.:

```
def objetosc( x, y ):
    z = 3
    return x + y + z

print( objetosc( 2, 4 ) )
print( objetosc( 2, 1 ) * 5 )

9
30
```

Zadanie 6

Utwórz funkcję o nazwie kwadrat(), która będzie miała **jeden parametr**. Funkcja ma zwracać podniesioną do kwadratu wartość przekazaną jej przez parametr.

Funkcja ma **zwracać wartość**, tzn. musisz w niej umieścić instrukcję return oraz odpowiednie wyrażenie (działanie) po tej instrukcji. Po wczytaniu boku kwadratu **wywołaj funkcję kwadrat() i wyświetl wynik na ekranie**.

Nazwy parametrów możesz sam wymyślić, trzymając się reguł Pythona związanych z nazwami zmiennych. Wynik działania programu powinien wyświetlić się w postaci widocznej poniżej:

```
===== REST
Podaj długość boku kwadratu: 7
Pole kwadratu wynosi: 49
|
```

Zadanie 7

Utwórz funkcję o nazwie predkosc(), która będzie miała dwa parametry. Przez pierwszy parametr będzie przekazywana do funkcji droga w metrach pokonana przez pojazd, przez drugi parametr czas pokonania tej drogi w sekundach. Funkcja powinna zwrócić prędkość w m/s.

```
| Wprowadz drogę pokonaną przez pojazd w metrach: 120
| Podaj czas jazdy w sekundach : 30
| Prędkość wyniosła 4 m/s
```

Zadanie 8

Obrotowy zraszacz nawadnia pole kukurydzy. Napisz funkcję obliczPole() z jednym parametrem, która zwróci pole nawadnianej przez zraszacz powierzchni w kształcie koła, jeżeli zasięg zraszacza przekazano przez parametr funkcji. Jeżeli będziesz potrzebować wartości liczby Pi, przyjmij $\pi = 3.1416$

```
Podaj wartość r: 4
Pole wynosi: 50.2656
```

Instrukcje warunkowe

Operatory porównania

Instrukcja warunkowa `if` służy do sterowania przebiegiem programu w zależności od wartości wyrażeń logicznych.

Po instrukcji `if` występuje wyrażenie logiczne.

Wyrażenie logiczne: wyrażenie, które ma wartość `True` (prawda) lub `False` (fałsz).

W wyrażeniach logicznych przeważnie porównujemy pewne wartości (np. wartości zmiennych). Do takich porównań możemy stosować operatory porównania:

```
> czy większe
< czy mniejsze
>= czy większe lub równe
<= czy mniejsze lub równe
== czy równe
!= czy różne
```

Np. wyrażenie:

```
2 == 0
```

ma wartość `False`, ponieważ dwa nie jest równe zero. Możesz się o tym przekonać wyświetlając wartość tego wyrażenia za pomocą funkcji `print()`:

```
print( 2 == 0 )
```

```
False
```

Zadanie:

wyświetl wartość wyrażenia `5 > 3`

```
print( 5 > 3 )
```

Operatory logiczne `and` i `or`

Tworząc w programie wyrażenia logiczne, możesz używać także operatorów logicznych: `and`, `or` i `not`.

Wyrażenie z użyciem operatora `and` ma wartość logiczną `True`, jeżeli wyrażenia po obu jego stronach mają wartość logiczną `True`.

Wyrażenia z operatorem `and`:

```
False and False → False
```

```
True and False → False
```

```
False and True → False
```

```
True and True → True
```

Wyrażenie z użyciem operatora `or` ma wartość logiczną `True`, jeżeli przynajmniej jedno z wyrażeń znajdujących się po obu jego stronach mają wartość logiczną `True`.

Wyrażenia z operatorem or:

False or False → False
 True or False → True
 False or True → True
 True or True → True

Operator not odwraca wartość logiczną następującego po nim wyrażenia.

Wyrażenia z operatorem not:

not False → True
 not True → False

Przykład E:

w trzech oddzielnych instrukcjach print() wyświetl wartości wyrażeń:

25 < 3 or 10 > 0
 3 == 3 and 5 < 6
 1 == 12 and 4 > -4

```
print( 25 < 3 or 10 > 0 )
print( 3 == 3 and 5 < 6 )
print( 1 == 12 and 4 > -4 )
```

```
True
True
False
```

Instrukcja warunkowa if

Dzięki instrukcji warunkowej if możesz wykonać blok instrukcji w zależności od tego, czy wyrażenie po if ma wartość True.

```
if warunek_logiczny:
    instrukcja_1
    instrukcja_2
    instrukcja_3
    ....
```

```
if 5 == 3:
    print( "Eureka!" )
    print( "Pięć jest równe
    trzy!!!" )

print( "To już jest koniec."
)
```

```
To już jest koniec.
```

5 nie jest równe 3, zatem wyrażenie po if ma wartość False i blok instrukcji po dwukropku został pominięty. Program przeszedł do kolejnej instrukcji po bloku, czyli:

```
print( "To już jest koniec."
)
```

Uwaga: zwróć uwagę, że operator porównania == (czy równe), składa się z dwóch znaków =. Nie myl go z **operatorem przypisania**, który składa się z jednego znaku =.

Zadanie 9

Zmodyfikuj funkcję czy5() w taki sposób, aby funkcja zwracała informację zwracaną przez parametr x. Pamiętaj, że instrukcja return kończy działanie funkcji. Np. funkcja:

```
def czy5( x ):
    if x == 5:
        return True
    return False
```

zwróci wartość True, jeżeli parametr x będzie miał wartość 5. W takim przypadku instrukcja return True zakończy działanie funkcji. Jeżeli parametr będzie miał wartość inną niż 5, program zrealizuje instrukcję return False i dopiero po niej wyjdzie z funkcji.

```
= RESTART: D:\Python\Python_gotowy
proszę wpisać liczbę x:
3
Warunek zwraca wartość:  False
|
```

Zadanie 10

Utwórz funkcję wieksza() w taki sposób, aby funkcja zwracała większą z wartości przekazanych do niej przez parametry x i y. Pamiętaj, że instrukcja return kończy działanie funkcji.

```
proszę wpisać liczbę x:
3
proszę wpisać liczbę y:
5
Liczba większa to 5
|
```

Sprawdzanie zakresu

W Pythonie można w prosty sposób badać czy wartość mieści się pewnym zakresie. Można w tym celu użyć wyrażenia z dwoma operatorami porównania, np.:

```
0 < x < 100
```

Takie wyrażenie będzie miało wartość True, jeżeli zmienna *x* ma wartość jednocześnie większą od zera i mniejszą od stu. Ten sam efekt można osiągnąć używając operatora logicznego *and* łączącego dwa osobne warunki:

```
x > 0 and x < 100
```

Zadanie 11

funkcja *dzien_tygodnia()* powinna zwracać wartość True jeżeli przez parametr *x* przekazano do niej wartość z zakresu od 1 do 7. W pozostałych przypadkach funkcja powinna zwracać wartość False.

```
----- RESTART: D:\1
Podaj liczbę od 1 do 7:4
True
```

Klauzula else

Instrukcja *if* pozwala na wykonanie bloku instrukcji także w przypadku, gdy badane wyrażenie ma wartość False. Służy do tego klauzula *else*.

```
if 5 == 3:
    print( "Eureka!" )
    print( "5 jest równe 3!!!" )
else:
    print( "Tak mi się wydawało," )
    print( "5 nie jest równe 3 :(" )

Tak mi się wydawało,
5 nie jest równe 3 :(
```

Warunek *5 == 3* ma wartość False, zatem został wykonany blok instrukcji po *else*. Zauważ, że po *else* wstawiamy dwukropek *:*, a blok instrukcji należący do klauzuli *else* tradycyjnie zaczyna się w nowej linii z wcięciem. Słowo kluczowe *else* jest umieszczone w tej samej odległości od prawej strony ekranu co instrukcja *if*.

Zadanie 12

Funkcja *kto_wygral()* zwraca imię zwycięzcy w konkurencji na pomiar prędkości. W zawodach biorą udział: Kojot i Struś. Prędkość Kojota przekazana jest przez parametr *vk*, prędkość Strusia przez parametr *vs*. Jeżeli prędkość Kojota jest większa od prędkości Strusia, funkcja powinna zwracać słowo **Kojot**. W pozostałych przypadkach funkcja powinna zwracać słowo **Strus** (bez polskich liter). Aby nieco utrudnić Ci życie, w zadaniu sprawdzimy dodatkowe warunki:

1. instrukcja *return* zwycięzca musi pozostać niezmienną
2. w funkcji może być tylko jedna instrukcja *return*
3. musisz użyć klauzuli *else*

```

1 def kto_wygral( vk, vs ):
2     zwyciezca = ""
3
4     return zwyciezca

```

```

==== RESTART: D:\Python\Pyt
Podaj prędkość Kojota: 23
Podaj prędkość Strusia: 56
Wygrał: Strus

```

Klauzula elif

Klauzule elif pozwalają w jednej instrukcji if badać wartości wielu wyrażeń. Np.

```

if 5 == 3:
    print( "Eureka!" )
    print( "5 jest równe 3!!!" )
elif 5 > 3:
    print( "5 jest większe od 3!!!" )
elif 5 == 5:
    print( "No tak, to jasne..." )
else:
    print( "Nic z tego nie" )
    print( "rozumiem..." )

```

5 jest większe od 3!!!

W jednej instrukcji if możesz użyć dowolnej liczby klauzul elif, jednak instrukcja if działa tylko do napotkania **pierwszego prawdziwego warunku**. Dlatego wykonany został kod po warunku $5 > 3$, a kolejna klauzula elif, pomimo tego, że warunek $5 == 5$ ma wartość True, nie jest już wykonywana.

Zadanie 13

Uzupełnij funkcję `dzien_tygodnia()` w taki sposób, aby zwracała odpowiedni łańcuch tekstowy w zależności od wartości przekazanej przez parametr `x` wg następujących kryteriów:

wartość x	zwracany string
1	poniedziałek
2	wtorek
3	środa
4	czwartek
5	piątek
6	sobota
7	niedziela
inna niż powyższe	błąd

Dodatkowe kryterium: w Twoim programie może być tylko jedna instrukcja return.

```
def dzien_tygodnia( x ):
    dzien = "poniedziałek"

    return dzien

Podaj dzień tygodnia :3
środa
```

Zadanie 14 Oczyszczacz powietrza

Do funkcji setPower(), przez **pierwszy** parametr przekazywana jest z czujnika wartość odczytu zanieczyszczenia powietrza pyłem zawieszonym **PM 2,5**, zaś przez **drugi** parametr wartość odczytu zanieczyszczenia powietrza pyłem zawieszonym **PM 10**.

Funkcja steruje mocą wentylatora w oczyszczaczu powietrza. Funkcja powinna zwracać poziom mocy wentylatora, będący liczbą całkowitą z zakresu **od 0 do 3**, zależną od indeksu jakości powietrza, zgodnie z tabelą poniżej. **Uwaga:** w przypadku, gdy indeks jakości powietrza dla pyłów PM 2,5 i PM 10 jest różny, funkcja powinna ustawić moc wentylatora **wg gorszego** z tych indeksów.

Indeks jakości powietrza	PM 2,5	PM 10	Moc went.
Bardzo dobry	0-12	0-20	0
Dobry	13-24	21-50	1
Umiarkowany	25-36	51-80	2
Zły	>36	>80	3

```
1 def setPower( pm25, pm10 ):
2     return 0

==== RESTART: D:\Python\Python_got(
Podaj wartość parametru PM25 = 21
Podaj wartość parametru PM10 = 10
wynik dobry, moc wentylatora = 1
```

Zadanie 15 Trójkąty

Utwórz funkcję sprawdz_trojkat() **z trzema parametrami**. Przez parametry zostaną do funkcji przekazane długości trzech stalowych prętów, które maszyna przycina losowo w rozmiarze od 1 do 10 cm. Funkcja powinna informować czy z pociętych prętów można zespawać trójkąt.

```

==== RESTART: D:\Python\Pytl
Podaj bok a = 1
Podaj bok b = 9
Podaj bok c = 1
Nie można zespawać trójkąta
>>

==== RESTART: D:\Python\Pytl
Podaj bok a = 3
Podaj bok b = 4
Podaj bok c = 5
Można zespawać trójkąt
>>

```

Zadanie 16

Utwórz funkcję `sortuj_liczby()` z **trzema parametrami**. Przez parametry zostaną do funkcji przekazane trzy dowolne liczby. Funkcja powinna wskazać liczbę największą, środkową i najmniejszą.

```

Podaj liczbę a = 45
Podaj liczbę b = 2
Podaj liczbę c = 56
Liczba największa = 56

Liczba środkowa = 45

Liczba najmniejsza = 2

```

Pętla while

Pętla `while` przypomina w działaniu instrukcję `if`. Kod zawarty w ciele pętli także wykonywany jest wtedy, gdy wartość wyrażenie logicznego po instrukcji `while` ma wartość `True`. Jednak w odróżnieniu od `if`, nie dzieje się to jednorazowo. Kod wykonywany jest tak długo, jak długo wyrażenie po `while` jest prawdziwe!

Z tego powodu, projektując pętlę, musisz zatroszczyć się, aby wyrażenie logiczne mogło zmienić wartość. W przeciwnym razie utworzysz pętlę nieskończoną. Np.:

```

x = 1
while x > 0:
    print( "Ta pętla nigdy się nie
    skończy..." )

Ta pętla nigdy się nie skończy...
Ta pętla nigdy się nie skończy...
Ta pętla nigdy się nie skończy...
...

```

Tak, ta pętla nigdy się nie skończy, ponieważ zmienna `x` ma ciągle wartość `1`, zatem wyrażenie `x > 0` ma zawsze wartość `True`!

```
while x >= 0:
    print( x, "ta pętla wkrótce się
skończy...")
    x = x - 1
```

```
3 ta pętla wkrótce się skończy...
2 ta pętla wkrótce się skończy...
1 ta pętla wkrótce się skończy...
0 ta pętla wkrótce się skończy...
```

Zadanie 17

Twoim celem jest napisanie programu, który obliczy, po ilu miesiącach odkładania kieszonkowego uda Ci się kupić nowy komputer gamingowy.

Specyfikacja:

1. Użytkownik wprowadza **cenę komputera** (np. 5000 zł).
2. Użytkownik wprowadza kwotę, którą **już posiada** (stan początkowy).
3. Użytkownik wprowadza stałą kwotę, którą jest w stanie **odkładać co miesiąc** (np. 200 zł).
4. Program ma za pomocą pętli while dodawać co miesiąc kwotę oszczędności do stanu konta, aż suma będzie równa lub większa od ceny komputera.
5. Na koniec program powinien wyświetlić, ile miesięcy trwało oszczędzanie i ile pieniędzy zostało "reszty".

Ile kosztuje Twój wymarzony komputer? (zł): 1200

Ile masz już odłożone? (zł): 230

Ile możesz odkładać co miesiąc? (zł): 100

Miesiąc 1: Masz już 330.00 zł

Miesiąc 2: Masz już 430.00 zł

Miesiąc 3: Masz już 530.00 zł

Miesiąc 4: Masz już 630.00 zł

Miesiąc 5: Masz już 730.00 zł

Miesiąc 6: Masz już 830.00 zł

Miesiąc 7: Masz już 930.00 zł

Miesiąc 8: Masz już 1030.00 zł

Miesiąc 9: Masz już 1130.00 zł

Miesiąc 10: Masz już 1230.00 zł

GRATULACJE! Uzbierałeś na komputer w 10 miesięcy.

Zostanie Ci jeszcze 30.0 zł na gry!

>

Zadanie 18

Wyobraź sobie, że programujesz sterownik w szklarni. Twoim zadaniem jest monitorowanie temperatury wewnątrz obiektu.

1. Program powinien w pętli while prosić o podanie aktualnej temperatury (użytkownik wpisuje ją ręcznie).
2. Jeśli temperatura mieści się w bezpiecznym zakresie (**powyżej 0°C i poniżej 30°C**), program wyświetla komunikat: „Temperatura w normie” i kontynuuje pracę.
3. Jeśli temperatura osiągnie lub przekroczy **30°C**, program ma wyświetlić ostrzeżenie: „ALARM! Temperatura za wysoka! Wyłączam system.” i natychmiast przerwać działanie pętli.
4. Dodatkowo, wpisanie liczby mniejszej lub równej **0** powinno bezpiecznie wyłączyć program (jako kod serwisowy).

Skrócone operatory

W programach bardzo często dokonujemy operacji na zmiennych, w taki sposób, że wynik operacji zapisujemy w tej samej zmiennej. Np.:

```
wiek = 5
wiek = wiek + 1
print( wiek )
```

6

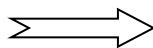
albo

```
cena = 40
cena = cena * 0.8
print( cena )
```

32

itp. Ponieważ te operacje są powszechne w programach, wymyślono skrócony sposób ich zapisu (podobnie jak w C++), tak, aby zmienna pojawiała się w nich tylko raz. Np. zamiast instrukcji

```
wiek = wiek + 1
```



```
wiek += 1
```

podobnie możesz sobie poradzić z drugim przykładem:

```
cena *= 0.8
```

operator	przykład instrukcji	instrukcja równoważna
+=	x += 1	x = x + 1
-=	x -= 1	x = x - 1
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
//=	x //= 5	x = x // 5
%=	x %= 2	x = x % 2

Alfabet

Wartości typu znakowego, którymi posługujesz się w Pythonie, pobierane są z tablicy **Unicode**, przechowującej tysiące różnych znaków. Pierwsze 128 znaków pokrywa się dokładnie z inną tabelą znaków: **ASCII**.

Słowniczek:

ASCII (*American Standard Code for Information Interchange*) - 7-bitowy system kodowania znaków w komunikacji elektronicznej. Przyporządkowuje liczbom z zakresu od 0 do 127 określone znaki, m.in. litery, cyfry, znaki interpunkcyjne.

Każdy znak jest zatem liczbą, której reprezentacja w postaci graficznej jest pobierana z odpowiedniego miejsca w tablicy Unicode.

Aby przekonać się, jaka pod jaką wartością liczbową zakodowany jest dany znak, możesz posłużyć się funkcją **ord()**, która zwraca tę wartość dla podanego w postaci znaku argumentu, np.:

```
print( ord( "A" ) )
```

```
65
```

albo

```
x = ord( "a" )
print( x )
```

```
97
```

Z kolei, aby sprawdzić jaki znak zakodowany jest pod określoną wartością, użyj funkcji **chr()** (podobnie jak w C++), która zwraca znak dla podanej wartości całkowitej. Np.:

```
print( chr( 66 ) )
```

```
B
```

albo

```
x = chr( 98 )
print( x )
```

```
b
```

Zadanie 19

Napisać program drukujący na ekranie kody ASCII dla znaków o numerach od 33 do 255. Program powinien zatrzymać się jeśli zapisze cały ekran (np. 20 wierszy) i czekać, aż wciśniemy dowolny klawisz. a także powinien drukować na ekranie trzy słupki kodów i odpowiadających mu znaków.

Uwaga! Zastosuj się do następujących zastrzeżeń:

1. W programie możesz użyć tylko jednej funkcji print()
2. Nie używaj pętli for - świetnie, jeśli ją znasz, ale tym razem poradź sobie przy pomocy pętli while
3. Twój kod nie może zawierać więcej niż 100 znaków.

```
===== RESTART: C:/Users/Ja/Desktop/Zadanie_19.py =====
kod znaku 33      !          kod znaku 34      "          kod znaku 35      #
kod znaku 36      $          kod znaku 37      %          kod znaku 38      &
kod znaku 39      '          kod znaku 40      (          kod znaku 41      )
kod znaku 42      *          kod znaku 43      +          kod znaku 44      ,
kod znaku 45      -          kod znaku 46      .          kod znaku 47      /
kod znaku 48      0          kod znaku 49      1          kod znaku 50      2
kod znaku 51      3          kod znaku 52      4          kod znaku 53      5
kod znaku 54      6          kod znaku 55      7          kod znaku 56      8
kod znaku 57      9          kod znaku 58      :          kod znaku 59      ;
kod znaku 60      <         kod znaku 61      =          kod znaku 62      >
kod znaku 63      ?          kod znaku 64      @          kod znaku 65      A
kod znaku 66      B          kod znaku 67      C          kod znaku 68      D
kod znaku 69      E          kod znaku 70      F          kod znaku 71      G
kod znaku 72      H          kod znaku 73      I          kod znaku 74      J
kod znaku 75      K          kod znaku 76      L          kod znaku 77      M
kod znaku 78      N          kod znaku 79      O          kod znaku 80      P
kod znaku 81      Q          kod znaku 82      R          kod znaku 83      S
kod znaku 84      T          kod znaku 85      U          kod znaku 86      V
kod znaku 87      W          kod znaku 88      X          kod znaku 89      Y
kod znaku 90      Z          kod znaku 91      [          kod znaku 92      \

Naciśnij Enter, aby kontynuować...|
```

1. Indeksowanie

W Pythonie są trzy struktury danych, które są **sekwencjami**, tzn. przechowują dane w postaci uporządkowanych szeregów wartości. Uporządkowane, tzn. takie, w których każda wartość jest na określonej pozycji. Dzięki temu możesz odwołać się do pojedynczej wartości w sekwencji używając numeru jej pozycji, czyli indeksu.

Indeks elementu sekwencji podaje się w nawiasach kwadratowych po samej sekwencji lub po nazwie zmiennej, która na tę sekwencję wskazuje, np.:

```
x = "ABCDE"
print( x[2] )
```

```
C
```

Podobnie jak w poprzednim zadaniu, nie musisz tworzyć żadnej własnej funkcji. Jeśli ją jednak stworzysz, musisz ją także wywołać, tak aby w konsoli wyświetliły się poprawne wartości.

Jak widzisz łańcuchy znaków są sekwencjami - składają się bowiem z uporządkowanego szeregu znaków, z których każdy ma swoją określoną pozycję. Elementy w sekwencjach numerowane są od 0 (zera), zatem element o indeksie 2 jest tak naprawdę trzecim elementem sekwencji. Dlatego w przykładzie `x[2]` to litera C. Ogólnie wygląda to tak:

Elementy możesz także numerować od prawej strony, używając ujemnych indeksów, tak jak na ilustracji powyżej, np.:

0	1	2	3	4	5	6	7
F	R	Y	D	E	R	Y	K
-8	-7	-6	-5	-4	-3	-2	-1

```
x = "ABCDE"
print( x[-1] )
```

```
E
```

Pętla for

Pętla `for` umożliwia iterowanie po elementach sekwencji, tzn. przechodzenie po kolejnych elementach sekwencji.

Działanie pętli `for` zobacz na przykładzie:

```
for x in "ABC":
    print( x )
```

```
A
B
C
```

W nagłówki pętli po słowie kluczowym `for` deklarujemy zmienną pomocniczą, do której, w każdej iteracji pętli przypisywany jest kolejny element sekwencji, którą umieszczamy po operatorze `in`. Zwróć uwagę także na dwukropek, po którym w nowej linii z wcięciem rozpoczyna się ciało pętli - czyli instrukcje wykonywane w każdej iteracji.

Jak widzisz w powyższym przykładzie, do zmiennej pomocniczej `x` przypisywane są kolejne znaki łańcucha `"ABC"`, a w ciele pętli są one kolejno wyświetlane.

Zadanie 20

Zadanie: addDollar

do funkcji addDollar() przekazano przez parametr value pewien łańcuch znaków. Funkcja powinna zwrócić łańcuch zmodyfikowany w taki sposób, aby po każdym znaku został umieszczony znak \$. **Przykład:**

Wejście: "ABCDE"

Wyjście: "A\$B\$C\$D\$E\$"

Wejście: "Monty"

Wyjście: "M\$o\$n\$t\$y\$"

Jak już wiesz, łańcuchy są niemodyfikowalne. Nie możesz do nich np. wstawić nowego znaku. Dlatego będziesz potrzebował stworzyć nowy łańcuch powstały ze znaków łańcucha przekazanego do funkcji uzupełnionych o znaki \$. Dobrym pomysłem może być utworzenie nowej zmiennej, która będzie wskazywała na nowy łańcuch. Na początek może być to pusty łańcuch, np.:

```
y=""
```

Następnie użyj pętli for, aby przejść po każdym znaku przekazanego łańcucha:

```
for x in value:
```

W każdej iteracji tej pętli x będzie kolejnym znakiem z value. Do łańcucha y potrzebujesz dodać ten znak oraz dodatkowy znak \$:

```
y = y + x + "$"
```

Można tę instrukcję zapisać krócej w ten sposób:

```
y += x + "$"
```

Na koniec funkcja powinna zwrócić zmienną y.

Wpisz tekst do zmodyfikowania: Barcelona
Tekst po modyfikacji: B\$a\$r\$c\$e\$I\$o\$n\$a\$

Funkcja range()

W pętli for możesz skorzystać z funkcji range(), która zwraca sekwencję liczb całkowitych. Funkcja przyjmuje od jednego do trzech argumentów. Podanie jednego argumentu powoduje, że funkcja zwraca wartości od 0 do argument minus jeden, np.:

```
for x in range( 3 ):
    print( x )
```

```
0
1
2
```

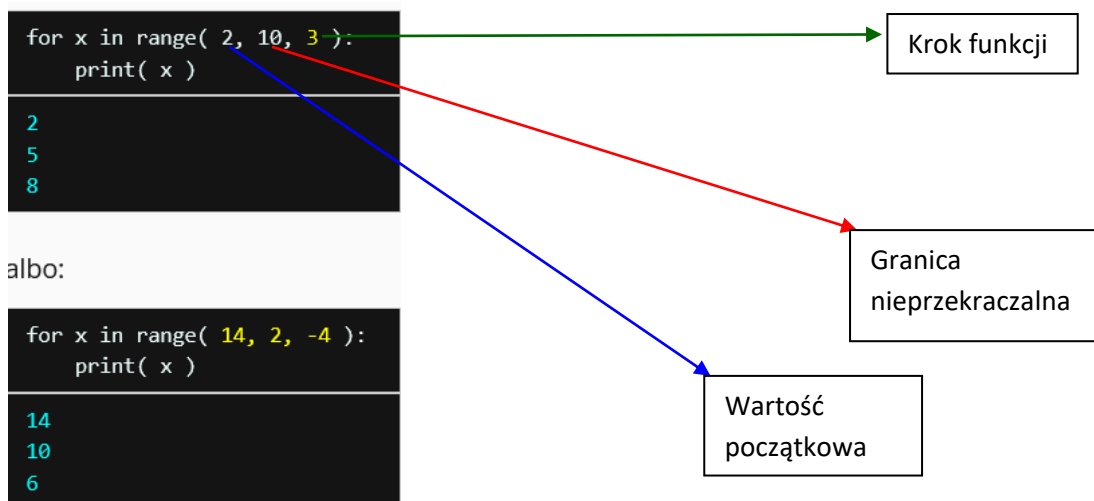
Jak widzisz, w każdej iteracji do zmiennej pomocniczej podstawiane są kolejne wartości zwracane przez funkcję range().

Podanie w funkcji range() dwóch argumentów powoduje, że funkcja zwraca wartości od pierwszego argumentu, do drugiego argumentu minus jeden, np.:

```
for x in range( 2, 5 ):
    print( x )
```

```
2
3
4
```

Podanie trzech argumentów powoduje, że funkcja range() zwraca wartości od pierwszego argumentu, zmierza do drugiego argumentu nie osiągając go, przy czym każda kolejna wartość powstaje przez dodanie trzeciego argumentu, np.:



Indeksowanie z range()

Zauważ, że wartości zwracanych przez funkcję range() możesz w łatwy sposób używać jako indeksów elementów sekwencji, np.:

```
wartosc = "ABC"
for x in range( len( wartosc
) ):
    print( wartosc[ x ] )
```

```
A
B
C
```

Wyrażenie `len( wartosc )` zwraca wartość 3, zatem w pierwszej iteracji pętli `x` ma wartość 0, w drugiej iteracji 1, a w trzeciej iteracji 2. Instrukcja:

```
print( wartosc[ x ] )
```

wyświetla zatem kolejne elementy łańcucha `wartosc`. Więcej zabawy z pętlą `for` czeka Cię przy okazji rozdziałów o kolejnych typach sekwencji :)

Zadanie 21

do funkcji **everyThird()** przez parametr **value** przekazano łańcuch, w którym zapisano wyniki pewnej liczby rzutów kością do gry. Funkcja powinna zwrócić łańcuch, składający się z co trzeciego zapisanego wyniku, zaczynając od pierwszego.

Przykład:

Wejście: "3441453221"

Wyjście: "3131"

Podaj wyniki kolejnych rzutów kostką (np. 123456): 123456123456
Podaję co trzeci wynik rzutów kostką: 1414

Przykład X

Napisz program do sprawdzania czy liczba wczytana z klawiatury jest liczbą pierwszą. Liczba pierwsza, to liczba naturalna $n > 1$, dla której istnieją tylko dwa dzielniki naturalne: 1 i n . Pamięamy też z zadań z C++ że, szukając liczby pierwszej: sprawdzamy czy kolejne liczby naturalne od 2 do pierwiastek(n) są podzielnikami liczby n . Sprawdzenie podzielności odbywa się poprzez użycie funkcji `%` (MOD→reszta z dzielenia), jeśli reszta z dzielenia jest zero to oznacza to, że liczba n jest podzielna przez liczbę mniejszą od n i n nie jest liczbą pierwszą.

```
# Pobieramy dane od użytkownika (input zwraca tekst, więc rzutujemy na int)
liczba = int(input("podaj liczbę naturalną, którą sprawdzę czy jest pierwsza="))

def czy_pierwsza(n):
    if n < 2:
        return False
# Sprawdzamy dzielniki od 2 do pierwiastka z liczby
# (Jeśli nie znajdziemy dzielnika do pierwiastka, to dalej też go nie będzie)
```

```

# liczba**0.5 to poniesie do potęgi 0,5 czyli pierwiastek 2 stopnia
# zapis int(liczba**0.5) + 1 gwarantuje nam ostatnia wartość iczba**0.5 też
zostanie sprawdzona

for i in range(2, int(n**0.5) + 1):
    if n % i == 0:
        return False

return True

if czy_pierwsza(liczba)==False:
    print(f"liczba {liczba} nie jest liczbą pierwszą ")
else:
    print(f"liczba {liczba} jest liczbą pierwszą ")

```

Zadanie 22

Z oparcia o powyższy program znajdź i zapisz w zeszycie liczbę pierwszą, która jest większą najbliższą lub równą liczbie $1000 + \text{nr_z_dziennika} * 100 + 15$

Np. nr 21

Liczba $= 1000 + 2100 + 15 = 3115$

Podaj numer z dziennika: 21

Najbliższa liczba pierwsza to: 3119

Zadanie 23

Napisać program, który po wczytaniu dwóch liczb naturalnych większych od 1, czyli liczba **dol** oraz liczba **gor** gdzie $\text{dol} < \text{gor}$. Wypisze sumę liczb pierwszych z tego przedziału.

Wskazówka:

Wykorzystaj poprzedni przykład. Utwórz pętlę dla zmiennej **j_naz** (z trzema literami nazwiska) od liczby **dol** do liczby **gor**. Utwórz zmienną **suma**. Pamiętaj o wyzerowaniu tej zmiennej. Użyj instrukcji **suma=suma + j_naz**. Po zakończeniu pętli wyprowadź zmienną **suma**. Przy wczytywaniu danych pamiętaj, że najmniejsza liczba pierwsza to 2.

Np.

Podaj dol=2

Podaj gor=10

Suma=17

Podaj dolną granicę: 23
 Podaj górną granicę: 1
 Dolna granica musi być mniejsza od górnej!
 Podaj dolną granicę: 2
 Podaj górną granicę: 10
 Liczba 2 to liczba pierwsza
 Liczba 3 to liczba pierwsza
 Liczba 5 to liczba pierwsza
 Liczba 7 to liczba pierwsza
 Suma liczb pierwszych = 17

Zadanie 24

"Cyber-klucz"

Pracujesz jako młodszy specjalista ds. cyberbezpieczeństwa. Twoja firma wykryła, że hakerzy ukryli tajne dane w plikach konfiguracyjnych. Kluczem do odszyfrowania wiadomości są tzw. **Liczby Pierwsze Bliźniacze**. Liczby bliźniacze to dwie liczby pierwsze, których różnica wynosi dokładnie 2 (np. 3 i 5, 5 i 7, 11 i 13).

Twoim zadaniem jest napisanie programu w Pythonie, który pomoże zespołowi bezpieczeństwa automatycznie generować i sprawdzać takie "cyber-klucze" w podanym przez użytkownika zakresie.

Wymagania funkcjonalne programu:

1. Program powinien poprosić użytkownika o podanie liczby całkowitej (np. zakresu, do którego szukamy liczb).
2. Skorzystaj z funkcji `czy_pierwsza(liczba)`, która zwraca `True`, jeśli liczba jest pierwsza, i `False` w przeciwnym wypadku. (Pamiętaj, że liczby mniejsze od 2 nie są pierwsze!).
3. Program ma przeszukać liczby od 2 do N i znaleźć wszystkie pary liczb bliźniaczych (czyli takich p oraz p+2, gdzie obie są pierwsze).
4. Na koniec program powinien wyświetlić te pary w czytelny sposób oraz podać, ile takich par udało się znaleźć.

```
--- SYSTEM GENEROWANIA CYBER-KLUCZY ---
```

```
Podaj górny zakres wyszukiwania (N): 34
```

```
Szukam par liczb bliźniaczych do 34...
```

```
Znaleziono cyber-klucz: (3, 5)
```

```
Znaleziono cyber-klucz: (5, 7)
```

```
Znaleziono cyber-klucz: (11, 13)
```

```
Znaleziono cyber-klucz: (17, 19)
```

```
Znaleziono cyber-klucz: (29, 31)
```

```
Operacja zakończona. Łącznie znaleziono par: 5
```